


```
mysterychap@MysteryChap:~$ sudo systemctl start vsftpd
mysterychap@MysteryChap:~$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable vsftpd
mysterychap@MysteryChap:~$ sudo systemctl status vsftpd
● vsftpd.service - vsftpd FTP server
   Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-02-25 13:27:05 UTC; 1min 9s ago
     Main PID: 1109 (vsftpd)
       Tasks: 1 (limit: 9330)
      Memory: 712.0K (peak: 1.5M)
         CPU: 11ms
        CGroup: /system.slice/vsftpd.service
               └─1109 /usr/sbin/vsftpd /etc/vsftpd.conf

Feb 25 13:27:05 MysteryChap systemd[1]: Starting vsftpd.service - vsftpd FTP server...
Feb 25 13:27:05 MysteryChap systemd[1]: Started vsftpd.service - vsftpd FTP server.
mysterychap@MysteryChap:~$
```

c) Create a backup of the original configuration file and configure the file

```
mysterychap@MysteryChap:~$ ls /etc/vsftpd*
/etc/vsftpd.conf
mysterychap@MysteryChap:~$ cp /etc/vsftpd.conf /etc/vsftpd.bak
cp: cannot create regular file '/etc/vsftpd.bak': Permission denied
mysterychap@MysteryChap:~$ sudo cp /etc/vsftpd.conf /etc/vsftpd.bak
mysterychap@MysteryChap:~$
```

```
mysterychap@MysteryChap:~$ sudo nano /etc/vsftpd.conf
mysterychap@MysteryChap:~$ sudo grep -v "^#" /etc/vsftpd.c
vsftpd.chroot_list vsftpd.conf
mysterychap@MysteryChap:~$ sudo grep -v "^#" /etc/vsftpd.conf
listen=NO
listen_ipv6=YES
anonymous_enable=NO
local_enable=YES
write_enable=YES
dirmessage_enable=YES
use_localtime=YES
xferlog_enable=YES
connect_from_port_20=YES
chroot_local_user=YES
chroot_list_enable=YES
secure_chroot_dir=/var/run/vsftpd/empty
pam_service_name=vsftpd
rsa_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
rsa_private_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
ssl_enable=NO
mysterychap@MysteryChap:~$
```

d) create a user particularly for FTP access

```
mysterychap@MysteryChap:~$ sudo adduser user2
info: Adding user `user2' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `user2' (1002) ...
info: Adding new user `user2' (1002) with group `user2 (1002)' ...
info: Creating home directory `/home/user2' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
```

e) Create required directory for the FTP access. Since we have enabled the chroot option in the configuration we have to change the home directory ownership to root. No other user must access that creating a security jail and have a folder accessible by ftp user.

```
mysterychap@MysteryChap:~$ sudo chown root:root /home/user2
mysterychap@MysteryChap:~$ sudo chmod 755 /home/user2
mysterychap@MysteryChap:~$ sudo mkdir -p /home/user2/ftp
mysterychap@MysteryChap:~$ sudo chown -R user2:user2 /home/user2/ftp
mysterychap@MysteryChap:~$ sudo chmod -R 755 /home/user2/ftp
```

f) Create files for transfer.

```
mysterychap@MysteryChap:~$ su user2
Password:
user2@MysteryChap:~/home/mysterychap$ cd
user2@MysteryChap:~$ cd ftp/
```

```
user2@MysteryChap:~/ftp$ cat > filesend2.txt
File in the ftp user directory to be sent to connectors directory
user2@MysteryChap:~/ftp$
```

g) Connect to the my machine with its IP address and login with the ftp username and password (check before remote machine connection)

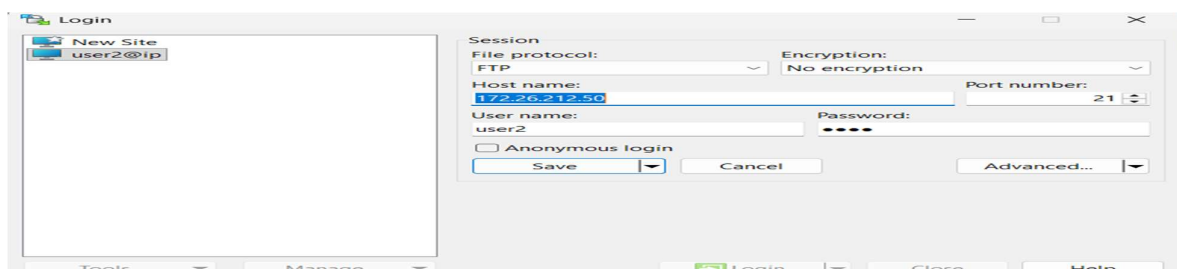
```
mysterychap@MysteryChap:~$ ftp 10.13.248.207
Connected to 10.13.248.207.
220 (vsFTPd 3.0.5)
Name (10.13.248.207:mysterychap): user2
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||13837|)
150 Here comes the directory listing.
drwxr-xr-x  2 1002      1002      4096 Feb 25 15:01 ftp
226 Directory send OK.
ftp> cd ftp
250 Directory successfully changed.
ftp> get filesend2.txt
local: filesend2.txt remote: filesend2.txt
229 Entering Extended Passive Mode (|||47437|)
150 Opening BINARY mode data connection for filesend2.txt (66 bytes).
100% |*****| 66 976.56 KiB/s 00:00 ETA
226 Transfer complete.
66 bytes received in 00:00 (114.68 KiB/s)
ftp> put file.sh
local: file.sh remote: file.sh
229 Entering Extended Passive Mode (|||60488|)
150 Ok to send data.
100% |*****| 104 105.35 KiB/s 00:00 ETA
226 Transfer complete.
104 bytes sent in 00:00 (78.18 KiB/s)
ftp> ls
229 Entering Extended Passive Mode (|||44199|)
150 Here comes the directory listing.
-rw-----  1 1002      1002      104 Feb 25 15:10 file.sh
-rw-rw-r--  1 1002      1002      66 Feb 25 15:06 filesend2.txt
226 Directory send OK.
ftp>
```

h) Remote Machine FTP: (Set firewall rules in case of windows system or use ufw to allow ports in case of linux on both the system or turn of the firewall temporarily till the configuration is complete)

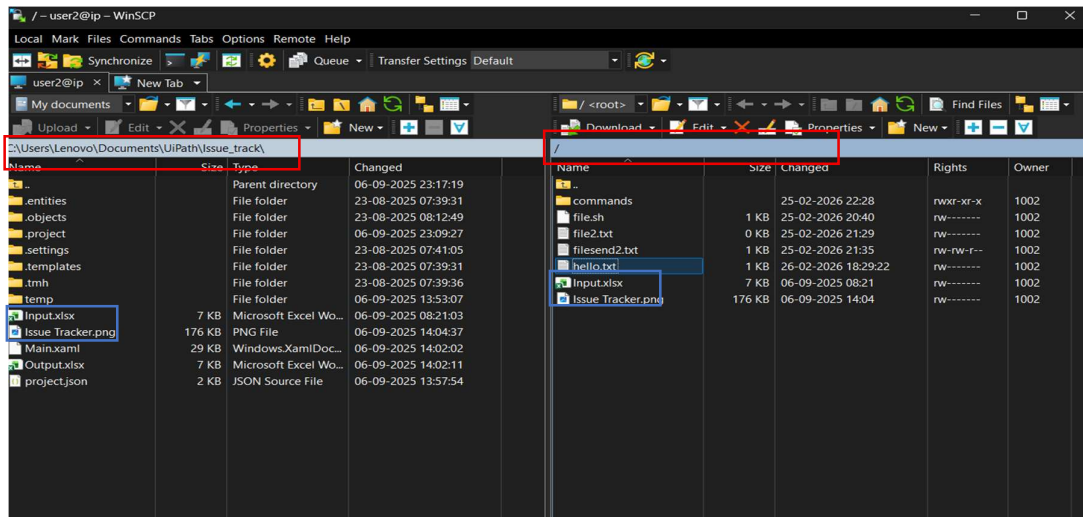
1. Check connectivity using ping (make sure they are in the same subnet or a static-ip with DNS resolution)

```
PS C:\Users\Lenovo> ping 172.26.212.50
Pinging 172.26.212.50 with 32 bytes of data:
Reply from 172.26.212.50: bytes=32 time=59ms TTL=128
Reply from 172.26.212.50: bytes=32 time=61ms TTL=128
Reply from 172.26.212.50: bytes=32 time=272ms TTL=128
Reply from 172.26.212.50: bytes=32 time=26ms TTL=128
Ping statistics for 172.26.212.50:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 26ms, Maximum = 272ms, Average = 104ms
PS C:\Users\Lenovo>
```

2. Since the remote system was a windows' one I used WinSCP application to connect to file server.



3. Connection established and file transferred



4. I have also captured the packet and saw the complete connection flow

The screenshot shows a Wireshark capture of an FTP session. The packet list on the left shows the flow of the connection, including the initial connection, directory listing, and file transfer. The packet details pane on the right shows the structure of the packets, including the `Request: TYPE I` and `Response: 200 Switching to Binary mode.`

No.	Time	Source	Destination	Protocol	Length	Info
57	13.303961	172.26.212.6	172.26.212.50	FTP	62	Request: TYPE I
58	13.304417	172.26.212.50	172.26.212.6	FTP	85	Response: 200 Switching to Binary mode.
59	13.312771	172.26.212.6	172.26.212.50	FTP	81	Request: PORT 172,26,212,6,242,221
60	13.313584	172.26.212.50	172.26.212.6	FTP	105	Response: 200 PORT command successful. Consider using PASV.
61	13.317855	172.26.212.6	172.26.212.50	FTP	71	Request: STOR Input.xlsx
65	13.331575	172.26.212.50	172.26.212.6	FTP	76	Response: 150 Ok to send data.
67	13.339081	172.26.212.6	172.26.212.50	FTP-DATA	1514	FTP Data: 1448 bytes (PORT) (STOR Input.xlsx)
68	13.339451	172.26.212.6	172.26.212.50	FTP-DATA	1514	FTP Data: 1448 bytes (PORT) (STOR Input.xlsx)
69	13.339451	172.26.212.6	172.26.212.50	FTP-DATA	1514	FTP Data: 1448 bytes (PORT) (STOR Input.xlsx)
70	13.339451	172.26.212.6	172.26.212.50	FTP-DATA	1514	FTP Data: 1448 bytes (PORT) (STOR Input.xlsx)
73	13.349765	172.26.212.6	172.26.212.50	FTP-DATA	782	FTP Data: 716 bytes (PORT) (STOR Input.xlsx)
78	13.351438	172.26.212.50	172.26.212.6	FTP	78	Response: 226 Transfer complete.
80	13.357063	172.26.212.6	172.26.212.50	FTP	86	Request: MDTM 20250906025103 Input.xlsx
81	13.358015	172.26.212.50	172.26.212.6	FTP	87	Response: 213 File modification time set.
82	13.368101	172.26.212.6	172.26.212.50	FTP	62	Request: TYPE A
83	13.369780	172.26.212.50	172.26.212.6	FTP	84	Response: 200 Switching to ASCII mode.
84	13.374653	172.26.212.6	172.26.212.50	FTP	81	Request: PORT 172,26,212,6,242,222
85	13.375841	172.26.212.50	172.26.212.6	FTP	105	Response: 200 PORT command successful. Consider using PASV.
86	13.380537	172.26.212.6	172.26.212.50	FTP	63	Request: LIST -a
90	13.401230	172.26.212.50	172.26.212.6	FTP	93	Response: 150 Here comes the directory listing.
91	13.401321	172.26.212.50	172.26.212.6	FTP-DATA	664	FTP Data: 598 bytes (PORT) (LIST -a)
96	13.414570	172.26.212.50	172.26.212.6	FTP	78	Response: 226 Directory send OK.

2.2) Configuring SFTP server that uses SSH in the Application layer along with FTP for encryption.

i) Install the openssh-server (this will install the additional required packages automatically)

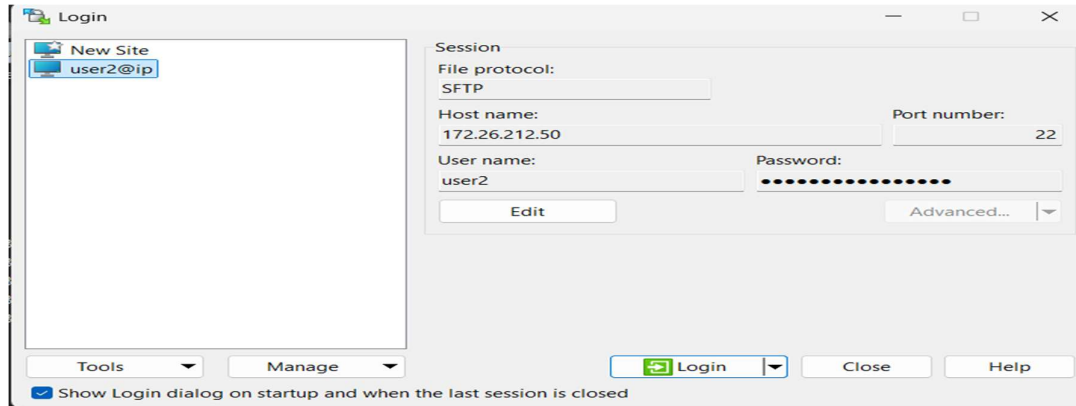
```
mysterychap@mysteryChap:~$ sudo apt install openssh-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libl1vm17t64
```

j) Now connect with the command 'sftp', send and receive the required files with put & get commands

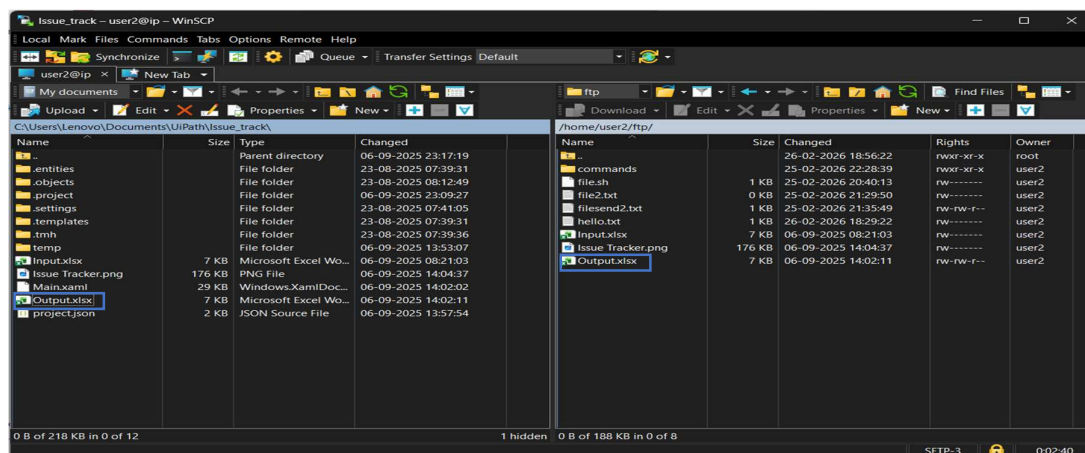
```
mysterychap@mysteryChap:~$ sftp user2@10.13.248.207
user2@10.13.248.207's password:
Connected to 10.13.248.207.
sftp> ls
file2.txt  file_send2.txt  hello.txt
sftp> get fi
file2.txt  file_send2.txt
sftp> get file.sh
File "/home/user2/ftp/file.sh" not found.
sftp> get file.sh
Fetching /home/user2/ftp/file.sh to file.sh
sftp> put fil
file.sh  file.txt  file.txt.bak  file2.txt  file_send2.txt  file_send2.txt
sftp> put file_send2.txt
Uploading file_send2.txt to /home/user2/ftp/file_send2.txt
file_send2.txt
```

k) Remote Machine FTP: (Set inbound rules and outbound rules in case of windows system or use ufw to allow ports in case of linux on both the system or turn of the firewall temporarily till the configuration is complete)

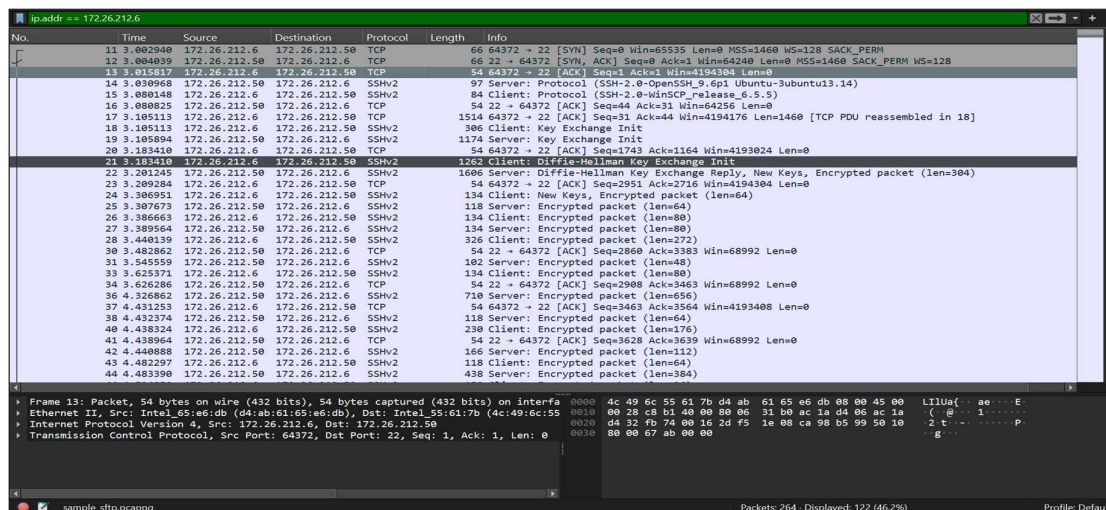
1. Similarly use WinSCP with SFTP option



2. Connection established and file transferred



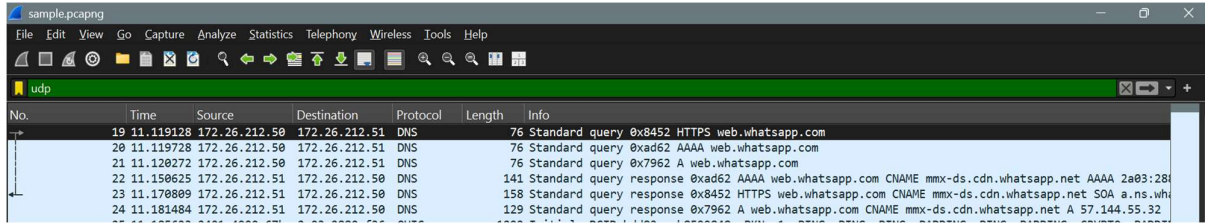
3. I have captured the packets for analysis for SFTP since it used SSH for connection instead of plain FTP and TCP for transfer the protocol filed didn't specify FTP as its application layer protocol



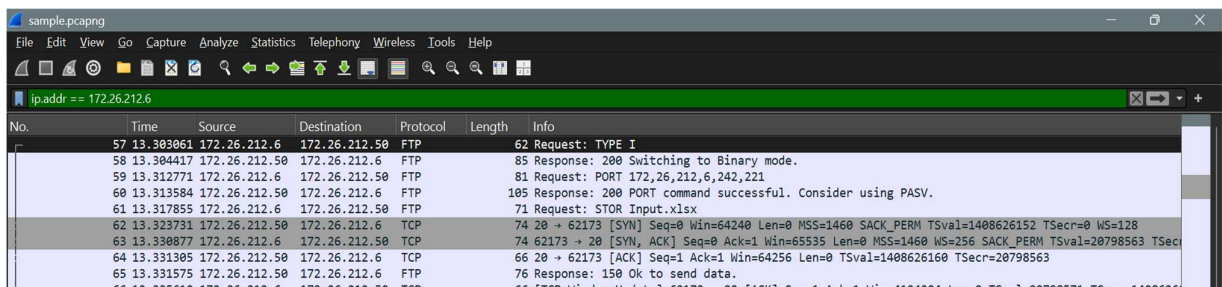
3. Explore with Wireshark/TCP-dump/Cisco Packet Tracer tools and learn about packet filters.

Packet filter with Wireshark (We can filter a packer by protocol/IP/port)

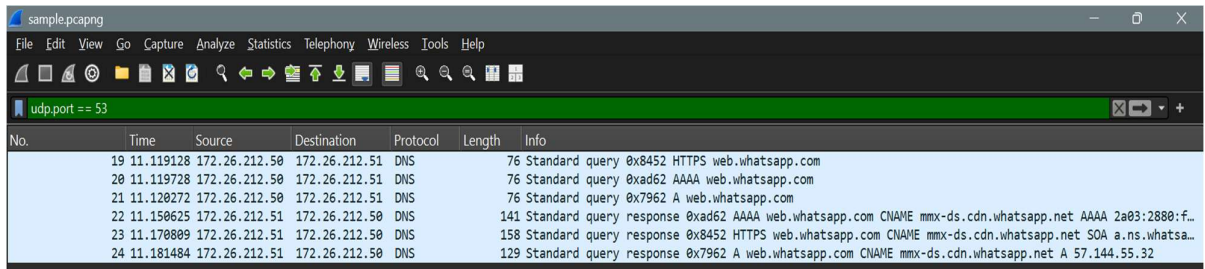
a) Using protocol



b) Using ip address



c) Using port



Packet Filter with TCPDump command (similarly we can filter by IP/Port/Protocol)

a) Using protocol

```
mysterychap@MysteryChap:/mnt/es$ tcpdump -r sample.pcapng -c 4 tcp
reading from file sample.pcapng, link-type EN10MB (Ethernet), snapshot length 262144
12:57:33.283072 IP 172.26.212.50.63624 > 20.195.84.7.https: Flags [P.], seq 535613594:535613625, ack 2487828336, win 253, length 31
12:57:33.332102 IP 172.26.212.50.62326 > 40.119.213.159.https: Flags [F.], seq 3415588682, ack 1824147525, win 254, length 0
12:57:33.381764 IP 20.195.84.7.https > 172.26.212.50.63624: Flags [F.], seq 1, ack 31, win 652, length 0
12:57:33.393193 IP 40.119.213.159.https > 172.26.212.50.62326: Flags [F.], seq 1, ack 1, win 63, length 0
```

b) Using ip address

```
mysterychap@MysteryChap:/mnt/es$ tcpdump -r sample.pcapng -c 4 host 172.26.212.50
reading from file sample.pcapng, link-type EN10MB (Ethernet), snapshot length 262144
12:57:33.283072 IP 172.26.212.50.63624 > 20.195.84.7.https: Flags [P.], seq 535613594:535613625, ack 2487828336, win 253, length 31
12:57:33.332102 IP 172.26.212.50.62326 > 40.119.213.159.https: Flags [F.], seq 3415588682, ack 1824147525, win 254, length 0
12:57:33.381764 IP 20.195.84.7.https > 172.26.212.50.63624: Flags [F.], seq 1, ack 31, win 652, length 0
12:57:33.393193 IP 40.119.213.159.https > 172.26.212.50.62326: Flags [F.], seq 1, ack 1, win 63, length 0
mysterychap@MysteryChap:/mnt/es$
```


c) Using port

```
mysterychap@MysteryChap:/mnt/e$ tcpdump -r sample.pcapng -c 4 port 53
reading from file sample.pcapng, link-type EN10MB (Ethernet), snapshot length 262144
12:57:44.402200 IP 172.26.212.50.61105 > 172.26.212.51.domain: 33874+ HTTPS? web.whatsapp.com. (34)
12:57:44.402800 IP 172.26.212.50.56995 > 172.26.212.51.domain: 44386+ AAAA? web.whatsapp.com. (34)
12:57:44.403344 IP 172.26.212.50.63635 > 172.26.212.51.domain: 31074+ A? web.whatsapp.com. (34)
12:57:44.433697 IP 172.26.212.51.domain > 172.26.212.50.56995: 44386 2/0/0 CNAME mmx-ds.cdn.whatsapp.net., AAAA 2a03:288
0:f36a:120:face:b00c:0:167 (99)
```

4. Understand linux utility commands like ping, arp (Understand each param from ifconfig output)

ping:

Ping is a Linux utility command that comes with the inet tools. It is used to check network connectivity between two devices or between a device and a domain name. Ping works using ICMP (Internet Control Message Protocol). It sends ICMP Echo Request packets to the destination and waits for ICMP Echo Reply packets in response. If replies are received (ICMP Echo Response), it confirms that the destination is reachable and the network connection is working properly. Additionally it measures the RTT and packet loss.

```
mysterychap@MysteryChap:/mnt/e$ ping embedur.ai
PING embedur.ai (162.159.137.54) 56(84) bytes of data:
64 bytes from 162.159.137.54: icmp_seq=1 ttl=55 time=35.4 ms
64 bytes from 162.159.137.54: icmp_seq=2 ttl=55 time=34.4 ms
64 bytes from 162.159.137.54: icmp_seq=3 ttl=55 time=36.2 ms
64 bytes from 162.159.137.54: icmp_seq=4 ttl=55 time=23.4 ms
^C
--- embedur.ai ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3262ms
rtt min/avg/max/mdev = 23.427/32.373/36.240/5.204 ms
```

arp (Address Resolution Protocol)

ARP is used to map an IP address to a physical MAC address in a local network. It maintains a table called the ARP cache, which stores the IP-to-MAC address mappings. It broadcasts the ARP request packet to all device in the network and once received the destination device unicast the ARP response packet to the sender with its MAC attached and commonly used among the devices in the same subnet.

```
mysterychap@MysteryChap:/mnt/e$ arp
Address          HWtype  HWaddress      Flags Mask    Iface
169.254.73.152   ether    00:11:22:33:44:55  CM           loopback0
mysterychap@MysteryChap:/mnt/e$ arp -a
? (169.254.73.152) at 00:11:22:33:44:55 [ether] PERM on loopback0
? (10.13.128.1) at 7c:5a:1c:cf:f2:a5 [ether] on eth1
mysterychap@MysteryChap:/mnt/e$ arp
Address          HWtype  HWaddress      Flags Mask    Iface
169.254.73.152   ether    00:11:22:33:44:55  CM           loopback0
10.13.128.1      ether    7c:5a:1c:cf:f2:a5  C            eth1
mysterychap@MysteryChap:/mnt/e$
```

ifconfig:

ifconfig displays and manages network interface configuration and statistics. We can also create a network interface or turn on or off the existing interface with ifconfig.

```
mysterychap@MysteryChap:/mnt/e$ ifconfig
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.13.248.207 netmask 255.255.128.0 broadcast 10.13.255.255
    inet6 fe80::3eaa:d41a:cd49:cb62 prefixlen 64 scopeid 0x20<link>
    ether 4c:49:6c:55:61:7b txqueuelen 1000 (Ethernet)
    RX packets 6003 bytes 380938 (380.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1558 bytes 124475 (124.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

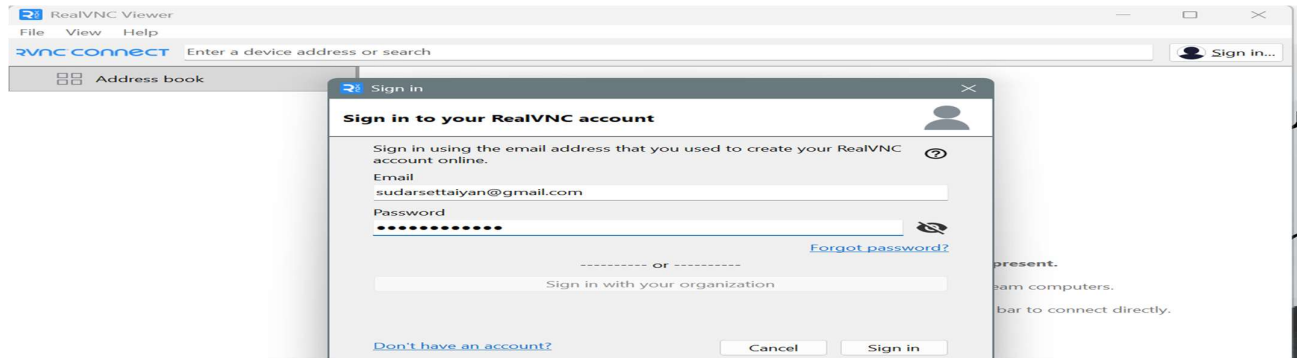
5. Understand what happens when duplicate IPs configured in a network.

When duplicate IPs are configured in a network the following things happen

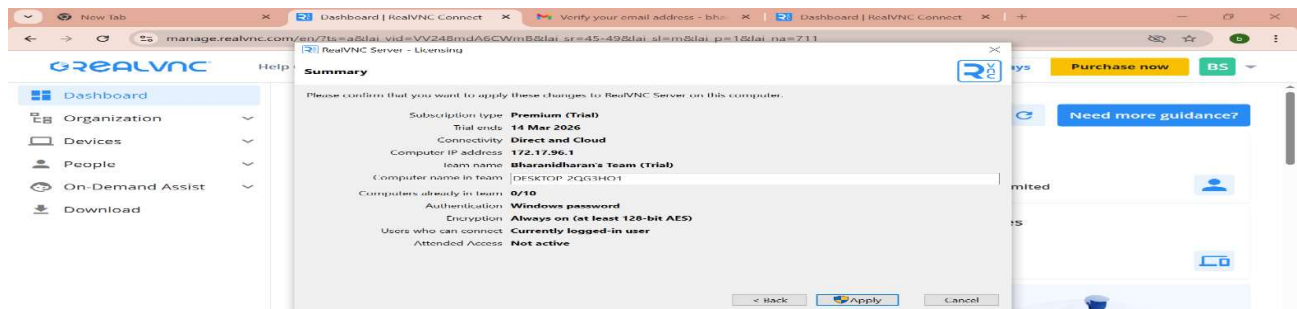
- a) Packet loss and unstable network behaviour.
- b) ARP resolution conflicts due to duplicate MAC-to-IP mappings.

6. Understand how to access remote system using VNC Viewer, Anydesk, Team Viewer and RDC.

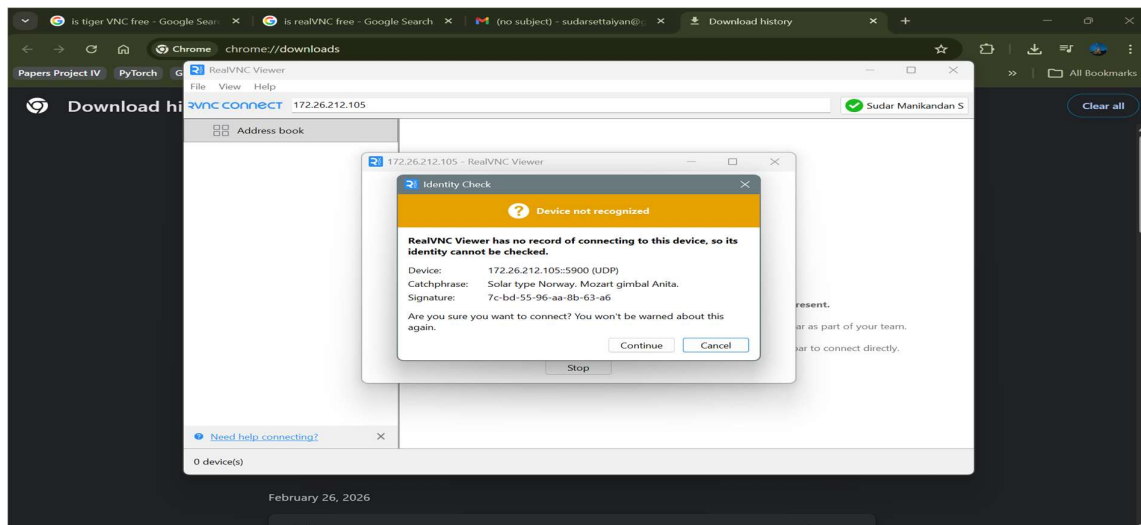
a) Install any VNC viewer in local system, VNC server on the remote system, create account and login



b) Now Configure the VNC server on the remote device as required (such as permission, usage limit etc...)



c) Now enter the IP of the remote device and connect in the client application. Once entered the remote device screen will be visible in the real VNC application



7. How to check your default gateway is reachable or not and understand about default gateway

Default Gateway:

A default gateway allows a system in a local network to connect with external subnet and Internet. Beyond simple routing, the gateway performs translation by repackaging data packets between different protocols and physical layers. It also uses Network Address Translation (NAT) to map private local IP addresses to a single public IP, ensuring seamless communication across diverse network structures.

```
mysterychap@MysteryChap:~$ ip r
default via 192.168.10.1 dev eth1 proto kernel metric 30
192.168.8.0/21 dev eth1 proto kernel scope link metric 286
192.168.10.1 dev eth1 proto kernel scope link metric 30
192.168.122.0/24 dev virbr0 proto kernel scope link src 192.168.122.1 linkdown
mysterychap@MysteryChap:~$ ip route
default via 192.168.10.1 dev eth1 proto kernel metric 30
192.168.8.0/21 dev eth1 proto kernel scope link metric 286
192.168.10.1 dev eth1 proto kernel scope link metric 30
192.168.122.0/24 dev virbr0 proto kernel scope link src 192.168.122.1 linkdown
mysterychap@MysteryChap:~$ netstat -r
Kernel IP routing table
Destination        Gateway            Genmask           Flags        MSS Window  irtt  Iface
default            192.168.10.1      0.0.0.0           UG           0 0        0     eth1
192.168.8.0        0.0.0.0           255.255.248.0     U            0 0        0     eth1
192.168.10.1       0.0.0.0           255.255.255.255   UH           0 0        0     eth1
192.168.122.0     0.0.0.0           255.255.255.0     U            0 0        0     virbr0
```

Then we can use ping command to check the connectivity.

```
mysterychap@MysteryChap:~$ ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data:
64 bytes from 192.168.10.1: icmp_seq=1 ttl=255 time=5.52 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=255 time=4.79 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=255 time=3.26 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=255 time=4.36 ms
^C
--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 300lms
rtt min/avg/max/mdev = 3.256/4.483/5.521/0.820 ms
```

8. Check iwconfig/ifconfig to understand in detail about network interface (check about interface speed, MTU and other parameters)

We can't check the speed of the network interface directly. So we use the ethtool commands to check the speed of the interface.

```
mysterychap@MysteryChap:~$ sudo ethtool eth0 | grep -i speed
Speed: 10000Mb/s
mysterychap@MysteryChap:~$ sudo ethtool eth1 | grep -i speed
Speed: 10000Mb/s
```

```
mysterychap@MysteryChap:~$ sudo ethtool virbr0 | grep -i speed
Speed: Unknown!
mysterychap@MysteryChap:~$ sudo ethtool loopback0 | grep -i speed
Speed: 10000Mb/s
mysterychap@MysteryChap:~$
```

```
mysterychap@MysteryChap:~$ ifconfig
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.10.141 netmask 255.255.255.0 broadcast 192.168.15.255
    inet6 fe80::3eaa:d41a:cd49:cb62 prefixlen 64 scopeid 0x20<link>
    ether 4c:49:6c:55:61:7b txqueuelen 1000 (Ethernet)
    RX packets 460 bytes 357710 (357.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 77 bytes 7080 (7.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 22 bytes 2614 (2.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 22 bytes 2614 (2.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

loppb0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    ether 00:15:5d:dd:e2:01 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

virbr0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
    ether 52:54:00:4f:f6:bf txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

9. Log in to your home router web interface and check the connected device list



10. Explain how DHCP server assigns IP address to device in your network.

The DHCP automatically assigns IP to the devices in the network using 4 steps. (DORA)

- Discovery: The client broadcasts a DHCPDISCOVER message to find available DHCP servers.
- Offer: DHCP servers receive the broadcast and respond with a DHCPOFFER, which includes an IP address, subnet mask, gateway, and lease duration.
- Request: The client receives the offer(s) and broadcasts a DHCPREQUEST, officially requesting the IP address from the chosen server.
- Acknowledgment: The server sends a DHCPACK to confirm the lease, allowing the client to start using the network.

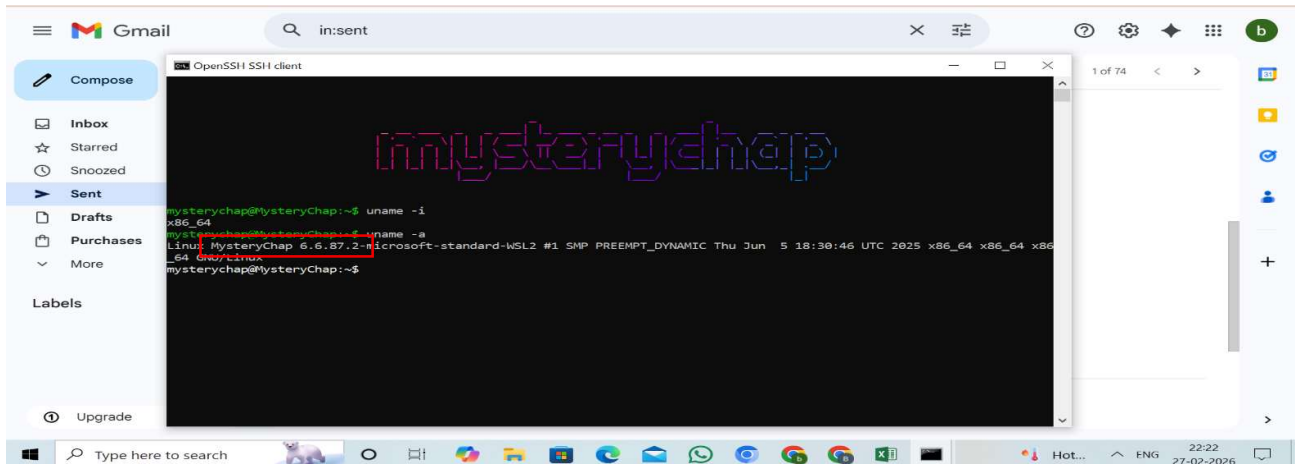
The client also checks the network for duplicate IP. If duplicate IP exist then the client sends DHCPDECLINE after ACK also during renewal the client may ask for its previous held IP, if that is already assigned to another device then the server will give NAK (Negative ACK). Apart from this if a client and server are different subnets then it uses a DHCP relay. I have also captured the packet for DHCP connection.

No.	Time	Source	Destination	Protocol	Length	Info
244	7.031344	10.11.241.234	10.11.128.1	DHCP	342	DHCP Release - Transaction ID 0x8f3c15c6
308	11.734073	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0xb96ac3f3
316	11.803110	10.11.128.1	10.11.241.234	DHCP	342	DHCP Offer - Transaction ID 0xb96ac3f3
317	11.805608	0.0.0.0	255.255.255.255	DHCP	362	DHCP Request - Transaction ID 0xb96ac3f3
318	11.839254	10.11.128.1	10.11.241.234	DHCP	342	DHCP ACK - Transaction ID 0xb96ac3f3

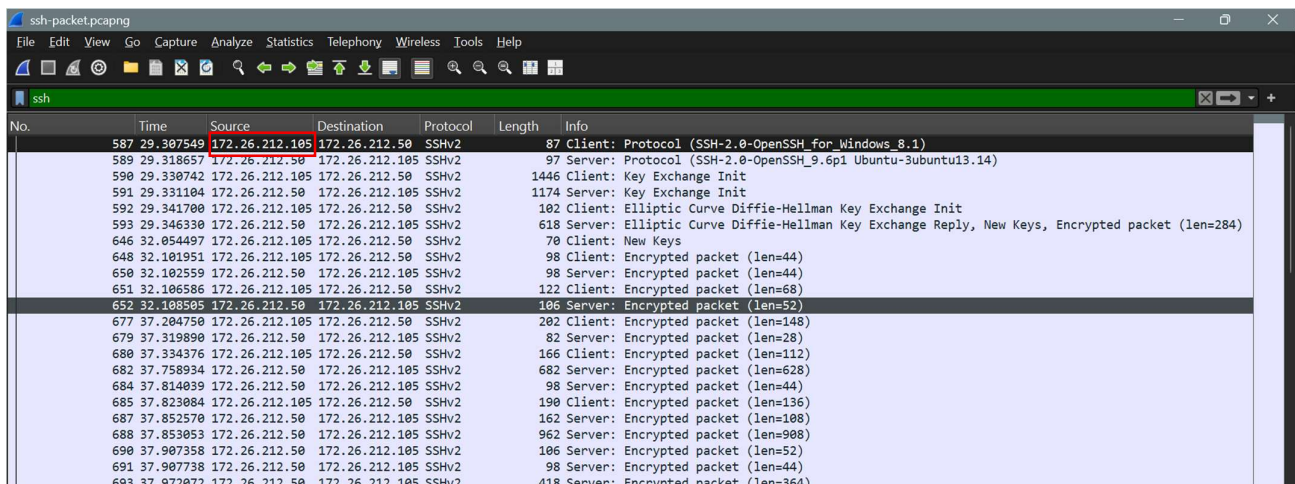
11. Use a terminal and connect to a remote machine using SSH and telnet.

Install and start the SSH server in the windows or linux in the host machine. Then connect the cline machine using the username and IP. For password less authentication we can set the public key of the client tot the trusted to known host.

Use command `ssh mysterychap@<ip>`. This will prompt the password. Once entered it will take to the following screen.

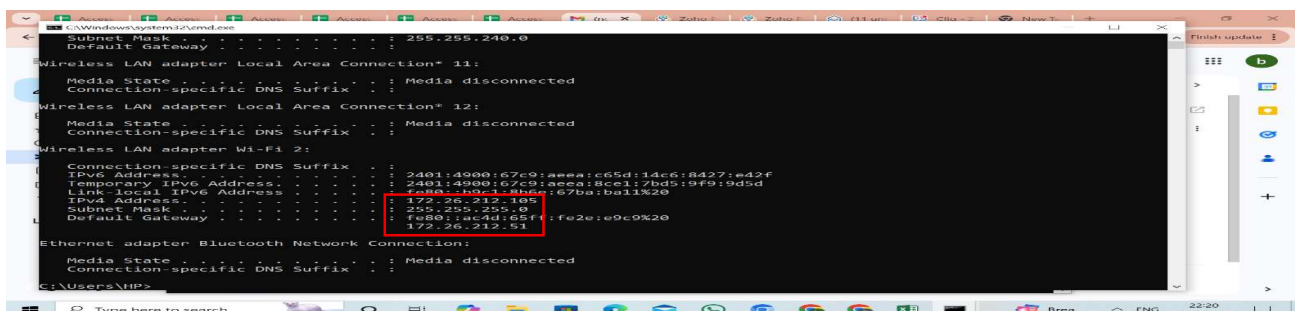


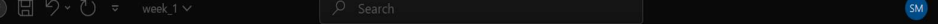
Here is the packet captured during the SSH connection between the client and server.



No.	Time	Source	Destination	Protocol	Length	Info
587	29.307549	172.26.212.105	172.26.212.50	SSHv2	87	Client: Protocol (SSH-2.0-OpenSSH_for_Windows_8.1)
589	29.318657	172.26.212.50	172.26.212.105	SSHv2	97	Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14)
590	29.330742	172.26.212.105	172.26.212.50	SSHv2	1446	Client: Key Exchange Init
591	29.331104	172.26.212.50	172.26.212.105	SSHv2	1174	Server: Key Exchange Init
592	29.341700	172.26.212.105	172.26.212.50	SSHv2	102	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
593	29.346330	172.26.212.50	172.26.212.105	SSHv2	618	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=284)
646	32.054497	172.26.212.105	172.26.212.50	SSHv2	70	Client: New Keys
648	32.101951	172.26.212.105	172.26.212.50	SSHv2	98	Client: Encrypted packet (len=44)
650	32.102559	172.26.212.50	172.26.212.105	SSHv2	98	Server: Encrypted packet (len=44)
651	32.106586	172.26.212.105	172.26.212.50	SSHv2	122	Client: Encrypted packet (len=68)
652	32.108505	172.26.212.50	172.26.212.105	SSHv2	106	Server: Encrypted packet (len=52)
677	37.204750	172.26.212.105	172.26.212.50	SSHv2	202	Client: Encrypted packet (len=148)
679	37.319890	172.26.212.50	172.26.212.105	SSHv2	82	Server: Encrypted packet (len=28)
680	37.334376	172.26.212.105	172.26.212.50	SSHv2	166	Client: Encrypted packet (len=112)
682	37.758934	172.26.212.50	172.26.212.105	SSHv2	682	Server: Encrypted packet (len=628)
684	37.814039	172.26.212.50	172.26.212.105	SSHv2	98	Server: Encrypted packet (len=44)
685	37.823084	172.26.212.105	172.26.212.50	SSHv2	190	Client: Encrypted packet (len=136)
687	37.852570	172.26.212.50	172.26.212.105	SSHv2	162	Server: Encrypted packet (len=108)
688	37.853053	172.26.212.50	172.26.212.105	SSHv2	962	Server: Encrypted packet (len=908)
690	37.907358	172.26.212.50	172.26.212.105	SSHv2	106	Server: Encrypted packet (len=52)
691	37.907738	172.26.212.50	172.26.212.105	SSHv2	98	Server: Encrypted packet (len=44)
693	37.972072	172.26.212.50	172.26.212.105	SSHv2	418	Server: Encrypted packet (len=364)

Here is the client system IP for verification.



- 
- The screenshot shows a Windows File Explorer window with a dark theme. The address bar displays the path `C:\Users\Mystery Chap>python -m http.server 80`. The main pane shows the output of the command: `Serving HTTP on :: port 80 (http://[::]:80/) ...`. The Command Prompt window is open, showing the command and its output.

-

- [illegible]